isthmus-ablation-core Manual

Voxel ablation and coupling core

June 16, 2026

Contents

1 isthmus-ablation-core

`isthmus-ablation-core` is a C++ voxel ablation and coupling core. It owns solid voxel state, tracks mass and material properties, supports standalone verification runs, and is being shaped so DSMC/SPARTA can call it through thin commands, fixes, and computes.

The project requires ISTHMUS. ISTHMUS provides surface reconstruction, surface-to-voxel mapping, and the shared TIFF import utility. The core is still more than an ISTHMUS wrapper: it owns voxel bookkeeping, local voxel-face ablation, generated geometries, exact-solution verification, and DSMC coupling state.

1.1 Quick Start

If DSMC and ISTHMUS are already installed, set their roots once:

```
export DSMC_ROOT=$HOME/dsmc
export ISTHMUS_ROOT=$HOME/isthmus
```

Then build a private DSMC/IAC executable from this repository:

```
make mpi
make test-dsmc
```

The executable is:

```
build-dsmc/bin/dsmc-iac
```

This does not edit the DSMC checkout. The build creates a disposable overlay in `build-dsmc/dsmc-overlay/`, where DSMC source files and IAC bridge commands are symlinked together for compilation.

Run a coupled example:

```
build-dsmc/bin/dsmc-iac \
  -in examples/dsmc-sphere-kinetic/in.dsmc-sphere-kinetic
```

For standalone-only work:

```
make standalone
make test-standalone
```

1.2 Current Capabilities

- Create a slab voxel model.
- Import a small ISTHMUS-backed TIFF stack as voxels.
- Generate a deterministic rough carbon TIFF sample for lightweight examples

and tests.

- Assign material density.
- Define a constant mass-loss source.
- Use explicit or mass-Courant timesteps.

- Apply local voxel ablation with delete-empty behavior.
- Use simple standalone loops with `variable`, `label`, `next`, and `jump`.
- Print aligned standalone stats with `iac_stats` and `iac_stats_style`.
- Write a CSV history file.
- Write VTU voxel files for visual inspection.
- Compute single-species ideal-gas kinetic-theory flux on ISTHMUS triangles.
- Read DSMC `nflux_incident` surface tallies and convert them to voxel mass

loss through ISTHMUS triangle ownership.

- Read DSMC surface-reaction tallies and convert CO-forming reactions into

carbon voxel mass loss.

- Build a private DSMC executable that calls the core through thin

command-style bridge files without modifying the DSMC checkout.

- Include shared input files with `include`.
- Verify tracked quantities against input-file exact expressions.
- Optionally build visual verification reports from test data.

1.3 Rebuild Docs

Documentation source lives in `docs/`. After changing command syntax, examples, concepts, or behavior, rebuild the single PDF manual:

```
make docs
```

The generated manual is:

```
docs/isthmus-ablation-core-manual.pdf
```

You can also call the local PDF builder directly:

```
python3 tools/build-docs-pdf.py
```

1.4 Current Run Type

The standalone executable currently prints:

```
# Standalone voxel ablation
```

Future ISTHMUS-backed runs should use a more specific title, such as:

```
# Coupled voxel/ISTHMUS surface ablation
```

1.5 First Example

```
units si

voxel_material carbon density 1800.0
voxel_create solid slab nx 8 ny 4 nz 4 dx 1.0e-6 material carbon

source q1 constant 1.8 units kg/m2/s
iac_timestep mass/courant 0.5 source q1
variable ablation_time equal 4.0e-3
variable keep internal 1

iac_stats 1
iac_stats_style step time nvox ndel mass mf vf front

voxel_dump hist solid history 1 output/slab-direct-ablation/history.csv
voxel_dump vox solid vtu 1 output/slab-direct-ablation/voxels_*.vtu select active scalar mf

label ablate-loop
iac_limit time ${ablation_time}
voxel_ablate solid source q1 policy local face xlo delete yes
iac_run 1
iac_continue time ${ablation_time} variable keep
if "${keep} > 0" then "jump SELF ablate-loop"
```

The regression wrapper in `tests/inputs/slab-direct-ablation/in.slab-direct-command.verify` includes this example and adds exact-solution checks with `iac_verify`.

2 Getting Started

The easiest DSMC/IAC workflow is:

- Install or build DSMC and ISTHMUS somewhere on your machine.
- Tell this repository where they are.
- Build a private DSMC executable inside this repository's build directory.

Your DSMC checkout is not modified. The build creates an overlay under `build-dsmc/dsmc-overlay/` and writes the executable to:

```
build-dsmc/bin/dsmc-iac
```

2.1 One-Time Shell Setup

If DSMC and ISTHMUS live in stable locations, put these in `~/.bashrc`, `~/.zshrc`, or your shell startup file:

```
export DSMC_ROOT=$HOME/dsmc
export ISTHMUS_ROOT=$HOME/isthmus
```

Then open a new terminal or source the file:

```
source ~/.zshrc
```

You can skip this and pass paths directly to `make` instead:

```
make mpi DSMC_ROOT=/path/to/dsmc ISTHMUS_ROOT=/path/to/isthmus
```

2.2 Build DSMC/IAC

From a fresh clone:

```
make mpi
make test-dsmc
```

The top-level `make` targets provide a SPARTA-style front end while CMake still handles dependency discovery and test registration underneath. The common DSMC machine targets are:

```
make mpi
make mac_mpi
make serial
make dsmc DSMC_MACHINE=<machine>
```

Use `make check-mpi` or `make check-mac_mpi` to build and run the DSMC-hosted test suite in one command.

CMake presets are also available on machines with CMake 3.20 or newer:

```
cmake --preset dsmc
cmake --build --preset dsmc
ctest --preset dsmc
```

2.2.1 University Of Kentucky MCC

On the Morgan Compute Cluster, the default CMake module is sufficient for the top-level `make` workflow:

```
module load cmake
```

Then point IAC at DSMC and ISTHMUS and build with DSMC's normal `mpi` machine target:

```
export DSMC_ROOT=$HOME/TylerStoffel/dsmc
export ISTHMUS_ROOT=$HOME/TylerStoffel/isthmus

make mpi
make test-dsmc
```

MCC's system Python 3.6 is enough for the normal build and verification tests. The optional PDF documentation and report targets may need additional local tools.

The DSMC/IAC build:

- builds `libisthmus_ablation_core.a`;
- creates `build-dsmc/dsmc-overlay/src`;
- symlinks DSMC source files into that overlay;
- symlinks the IAC bridge command files into that overlay;
- generates the overlay `Makefile.package`;
- runs DSMC's normal `make <machine>` build there;
- writes `build-dsmc/bin/dsmc-iac`.

The default DSMC machine target is `mpi`. Override it with:

```
make mac_mpi
```

or:

```
make dsmc DSMC_MACHINE=mac_mpi
```

2.3 Run A DSMC/IAC Case

```
build-dsmc/bin/dsmc-iac \
  -in examples/dsmc-sphere-kinetic/in.dsmc-sphere-kinetic
```

`build-dsmc/bin/dsmc-iac` quiets native SPARTA screen output by default and prints compact IAC coupled summaries instead. Full SPARTA output is still written to the log file. Add `-iac-dsmc-verbose` to show native SPARTA screen output while debugging.

Compact `[IAC]` and `[SPA]` lines are colored by default when stdout is a terminal. IAC lines are green and SPARTA lines are blue. Control this with:

```
build-dsmc/bin/dsmc-iac --iac-color auto -in examples/.../in.case
build-dsmc/bin/dsmc-iac --iac-color on -in examples/.../in.case
build-dsmc/bin/dsmc-iac --iac-color off -in examples/.../in.case
```

Color is only applied to terminal output; log files remain plain text.

Run the fast DSMC verification suite:

```
make test-dsmc
```

2.4 Standalone Only

The standalone binary does not need DSMC:

```
make standalone
make test-standalone
```

The equivalent CMake preset workflow is:

```
cmake --preset standalone
cmake --build --preset standalone
ctest --preset standalone
```

Run the first standalone example:

```
./build/ia-core -in examples/slab-direct-ablation/in.slab-direct-ablation
```

2.5 Visual Outputs

Examples write under `output/` by default. For the direct slab example:

```
output/slab-direct-ablation/history.csv
output/slab-direct-ablation/voxels_000000.vtu
output/slab-direct-ablation/voxels_000001.vtu
...
```

Open VTU/VTP files directly in ParaView. The examples usually write active voxels only and select `mf` as the first visible cell scalar.

2.6 Rebuild The Manual

```
cmake --build --preset docs
```

The PDF is written to:

```
docs/isthmus-ablation-core-manual.pdf
```

3 Architecture

The project has two intended front doors into one shared core:

- Standalone mode: this repository owns parsing, timestep advancement, stats,

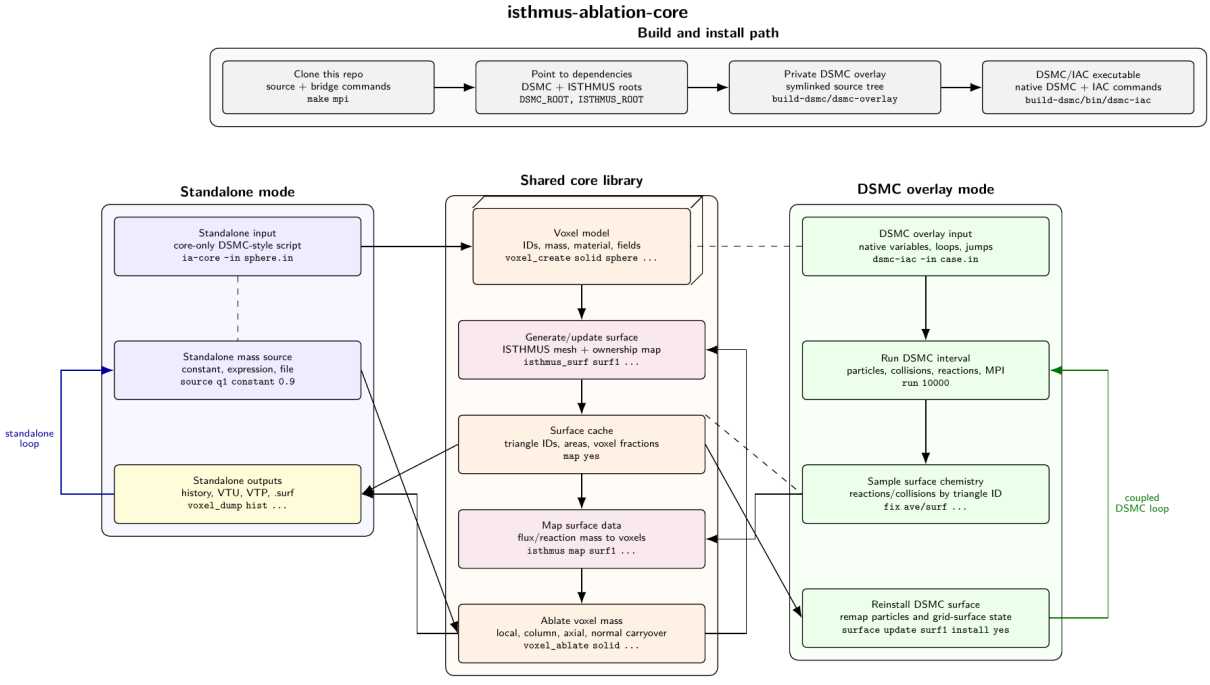
dumps, and verification.

- DSMC/SPARTA-linked mode: DSMC/SPARTA owns the main input parser and timestep

loop, while this project contributes commands, computes, and a core library object.
The shared core owns:

- voxel state with stable IDs;
- material density and voxel mass;
- generated and imported geometries;
- mass-loss sources;
- ablation policies;
- diagnostics and history;
- ISTHMUS surface state and ownership mapping.

The current flowchart is available here:



Architecture flowchart

Architecture flowchart

3.1 ISTHMUS Coupling

The current standalone ISTHMUS path is:

- Convert active voxels to an `isthmus::VoxelSet`.
- Call `isthmus::MarchingWindows`.
- Cache the surface mesh and triangle-to-voxel ownership fractions.
- Run `surf_flux ...` to apply mass flux to selected triangles.
- Run `voxel_ablate ...` to update voxel mass and delete empty voxels.
- Run `isthmus_surface ...` again if the voxel state changed.

3.2 Future DSMC/SPARTA Coupling

The planned DSMC coupling should support both a command-loop path and a iac_timestep-callback path. We will implement whichever proves easiest first.

The command-loop path would look like:

```
label ablate-loop
run 10000
surf_flux surf1 source dsmc-flux select all
voxel_ablate solid surface surf1 policy local delete yes
isthmus_surface surf1 voxels solid map yes
surface update surf1 install yes
jump SELF ablate-loop
```

The loop keeps DSMC/SPARTA in charge of particles, reactions, variables, labels, jumps, MPI, and stats. This project supplies commands that operate on the shared voxel/surface core between DSMC run intervals.

The callback path would expose similar core operations through a fix-style or other timestep callback so geometry can update during a longer run. That may be more natural for cases that need frequent ablation updates.

The main tradeoff is timing control. Command loops are explicit and fit native DSMC/SPARTA `label`, `jump`, and chunked `run` workflows. Callback coupling can hide more machinery behind a timestep hook, but may be more convenient for fine-grained updates.

4 Verification

Verification should be input-driven, not hardcoded in the core model.

The core tracks actual quantities such as:

- `mass`
- `mf`
- `vf`
- `front`
- `nvox`
- `ndel`

The input file declares exact or reference expressions:

```
iac_verify mf exact "1.0 - q1*time/(rho*length)" tolerance 0.01 percent norm max
iac_verify vf exact "1.0 - q1*time/(rho*length)" tolerance 0.01 percent norm final
iac_verify front exact "q1*time/rho" tolerance 0.01 percent norm final
iac_verify          rad exact "rad0 - q1*time/rho" tolerance 3.0 percent norm final
```

This keeps geometry-specific exact solutions out of the source code.

`mf` is the smooth mass ledger normalized by initial mass. `vf` is the active voxel count normalized by the initial active voxel count. It changes only when voxels are deleted, so it captures the stair-stepped voxelized geometry response.

4.1 Norms

`norm final` checks only the last history row.

`norm max` checks the maximum absolute error over recorded history rows.

`norm rms` checks the root-mean-square error over recorded history rows.

Regression tests should prefer percent tolerances unless an absolute error scale is more meaningful.

4.2 Convergence

Convergence checks are input-driven too. A test can define variables, use them inside the case setup, and ask the executable to rerun the same input with overrides:

```
variable resolution equal 20
variable steps equal 4
voxel_create solid sphere diameter 1.0e-3 resolution ${resolution} material carbon
variable i loop ${steps}

convergence         rad exact "rad0 - q1*time/rho" tolerance 100.0 percent norm final vary resolution 5 10 20 vary st
convergence vf exact "((rad0 - q1*time/rho)/rad0)^3" tolerance 100.0 percent norm final vary resolution 5 10 20 vary
```

This keeps convergence criteria in the test input instead of a separate helper script.

5 Expected Features

This page tracks planned user-visible capabilities. It is intentionally more stable than planning notes and brainstorm documents.

5.1 Voxel Core

- Generate slab, sphere, cylinder/cone, and imported voxel geometries.
- Track stable voxel IDs, lattice coordinates, material, remaining mass,

activity, fixed state, and future fields.

- Support local ablation, column carryover, axial carryover, normal-directed

carryover, ghost voxels, and configurable boundary behavior.

- Dump history, VTU voxel state, restart state, and diagnostic summaries.

5.2 Ablation Control

The long-term design should support both explicit commands and timestep callbacks. We will implement whichever path is easiest first while keeping the core API usable by both.

The command-oriented form is useful for readable standalone scripts and DSMC/SPARTA loops:

```
label ablate-loop
voxel_ablate solid source q1 policy local face xlo delete yes
isthmus_surface surf1 voxels solid map yes
surf_flux surf1 source dsmc-tallies select all
voxel_ablate solid surface surf1 policy local delete yes
jump SELF ablate-loop
```

Standalone inputs can loop over these commands directly. DSMC/SPARTA inputs can use native variables, labels, jumps, and repeated `run` chunks to couple gas evolution and voxel/surface updates.

The callback form may use a fix-style or other timestep hook to call the same core operations during a longer DSMC/SPARTA run.

5.3 ISTHMUS Coupling

- Generate a surface from active voxels using ISTHMUS.
- Cache triangle IDs, areas, normals, and triangle-to-voxel ownership

fractions.

- Map surface quantities back to voxel mass loss.
- Regenerate the surface when voxel state changes.
- Export VTP and SPARTA `.surf` files in standalone mode.
- Install or replace explicit DSMC/SPARTA surfaces in coupled mode.

5.4 DSMC/SPARTA Coupling

- Let DSMC/SPARTA own particle advancement, collisions, reactions, MPI,

variables, loops, and stats.

- Read per-surface DSMC tallies after a `run` interval.
- Map those tallies to ISTHMUS triangles and then to voxels.
- Apply voxel mass loss with chosen policies.
- Regenerate and reinstall the surface before the next `run` interval.

6 Command Reference

The command language is intentionally close to DSMC/SPARTA style: one command per line, simple positional arguments, keyword/value options, and underscore separated top-level command names. IAC commands use explicit prefixes such as `iac_`, `voxel_`, `isthmus_`, and `surf_` so they can live beside native DSMC commands like `create_grid`, `surf_modify`, `stats_style`, and `run`.

Current IAC commands:

`include` (include.md)

and `voxel_ghost` (voxel.md)

`source` (source.md)

`iac_timestep` (timestep.md)

`el`, `next`, and `jump` (loops.md)

`voxel_ablate` (voxel-ablate.md)

`isthmus_surface` (isthmus-surface.md)

d `surf_write_vtp` (surface.md)

`tu` and `grid_dump` (grid-write-vtu.md)

`inue`, and `iac_set` (iac.md)

`iac_stats_style` (stats.md)

`voxel_dump` (voxel-dump.md)

`iac_verify` (verify.md)

`iac_run` (run.md)

7 Input Language Map

The input language is split by ownership. IAC commands use explicit prefixes so they can sit beside native DSMC/SPARTA commands without ambiguity.

7.1 Command Families

| Family | Owns | Typical commands | | — | — | — | | `voxel_*` | Solid voxel state, material assignment, voxel dumps, mass removal | `voxel_material`, `voxel_create`, `voxel_ghost`, `voxel_ablate`, `voxel_dump` | | `isthmus_*` | ISTHMUS reconstruction from voxels | `isthmus_surface` | | `surf_*` | IAC surface flux, surface installation, surface output, DSMC surface measurements | `surf_flux`, `surf_install`, `surf_measure_flux`, `surf_dump`, `surf_write_vtp` | | `iac_*` | IAC solid time, stats, verification, bridge variables | `iac_timestep`, `iac_limit`, `iac_run`, `iac_continue`, `iac_verify`, `iac_stats` | | Native DSMC/SPARTA | Gas domain, particles, DSMC timestepping, collision/reaction models | `create_grid`, `species`, `mixture`, `surf_modify`, `run`, `timestep`, `stats` |

7.2 Time Advancement

In standalone mode, `iac_run` is the only timestep advancement command. In DSMC-hosted mode, native `run` advances particles and collisions, while `iac_run` advances only the solid voxel ledger.

Long commands can be split across physical lines with a trailing `&`:

```
surf_flux           skin dsmc/reaction fix rco column 1 sample-steps 100 &
                      reaction carbon-co.surf mass-courant 1.0 &
                      select normal nx 1.0 ny 0.0 nz 0.0 min-cos 0.5
```

The standalone parser joins these lines before tokenizing, matching DSMC/SPARTA input syntax.

The usual solid update order is:

```
iac_limit           time ${ablation_time}
isthmus_surface     skin voxels solid buffer 1 weighting no map yes
surf_flux           skin source q1 select all
voxel_ablate        solid surface skin policy carryover/normal delete yes
iac_run             1
```

For coupled DSMC, insert native DSMC sampling before the surface flux command:

```
fix                 sflux ave/surf all 1 100 100 c_sflux[*] ave one
run                 100 post no
surf_flux           skin dsmc/surf fix sflux mass-courant 0.1666666667
```

7.3 Portable Time Loops

Use this form when an input should run both standalone and through the DSMC bridge:

```
variable            ablation_time equal 4.0e-3
variable            keep internal 1
label               ablate-loop

iac_limit           time ${ablation_time}
voxel_ablate        solid source q1 policy local face xlo delete yes
iac_run             1

iac_continue        time ${ablation_time} variable keep
if                  "${keep} > 0" then "jump SELF ablate-loop"
```

The standalone parser recognizes this compact DSMC-style loop pattern. DSMC executes it with its normal variable, `if`, and `jump` machinery.

7.4 Verification Inputs

Examples should be runnable and visual by default. Tests should usually include an example, turn off expensive dumps, and add criteria:

```
include             examples/slab-direct-ablation/in.slab-direct-ablation

voxel_dump          off
surf_dump           off

iac_verify          mf exact "1.0 - q1*time/(rho*length)" tolerance 0.01 percent norm max
```

This keeps examples and tests from drifting apart while leaving examples useful for visual inspection.

8 `include` Command

The `include` command reads another input file at the point where the command appears. This keeps examples and tests from duplicating the same setup.

8.1 Syntax

```
include <path>
```

8.2 Examples

```
include ../../../examples/slab-direct-ablation/in.slab-direct-ablation
```

8.3 Description

Relative paths are resolved from the directory of the file that contains the `include` command.

The command is useful for regression tests:

```
include ../../../examples/slab-direct-ablation/in.slab-direct-ablation

iac_verify mass exact "mass0 - q1*area*time" tolerance 0.01 percent norm max
iac_verify mf exact "1.0 - q1*time/(rho*length)" tolerance 0.01 percent norm max
iac_verify front exact "q1*time/rho" tolerance 0.01 percent norm final
```

The example owns the physical case. The test wrapper owns the pass/fail criteria.

9 `voxel` Command

The `voxel` command family creates and configures voxel-state objects.

9.1 Syntax

```
voxel_material <name> density <rho> [molar-mass <kg/mol>] [formula <formula>]
voxel_create <model> slab nx <nx> ny <ny> nz <nz> dx <dx> material <name>
voxel_create <model> sphere diameter <D> dx <dx> material <name>
voxel_create <model> sphere diameter <D> resolution <N> material <name>
voxel_create <model> tiff file <path> dx <dx> material <name> \
  [ox <x>] [oy <y>] [oz <z>]
voxel_write_history <model> <path>
```

9.2 Examples

```
voxel_material carbon density 1800.0 molar-mass 0.0120107 formula C
voxel_create solid slab nx 8 ny 4 nz 4 dx 1.0e-6 material carbon
voxel_create solid sphere diameter 1.0e-3 dx 5.0e-5 material carbon
voxel_create solid sphere diameter 1.0e-3 resolution 20 material carbon
voxel_create solid tiff file examples/tiff-sphere/sphere-24.tif dx 5.0e-5 material carbon
voxel_ghost solid axis y boundary infinite layers 1
voxel_write_history solid output/dsmc-sphere-kinetic/history.csv
```

9.3 Description

`voxel_material` defines a material and its density. `molar-mass` and `formula` are optional for purely mechanical voxel ablation, but are used by DSMC-coupled surface-flux commands that infer solid mass consumption from a SPARTA surface-reaction file.

`voxel_create` creates a named voxel model. Each voxel receives a stable ID, integer lattice indices, a centroid, an initial mass based on density and `dx^3`, and an active flag.

`slab` fills the full rectangular lattice.

`sphere` fills voxel centroids inside a sphere with diameter `D`. Specify either `dx` or `resolution`, but not both. `resolution <N>` means `N` voxels across the sphere diameter, so `dx = D/N`. Voxels outside the sphere remain in the bounding lattice as inactive cells so the structured indexing and voxel IDs stay stable while the sphere ablates.

`tiff` imports a multipage TIFF stack through ISTHMUS's TIFF utility. The current shared utility supports a narrow uncompressed 8-bit grayscale stack convention, with voxels whose image value is `1` treated as active. The active coordinates returned by ISTHMUS are expanded into a structured IAC voxel lattice. `ox`, `oy`, and `oz` optionally shift the voxel-centroid origin.

`voxel_ghost` configures ghost voxel images used during surface generation. The first supported boundary is `boundary infinite`, which extrapolates active boundary voxels beyond the requested side of the requested axis. The optional `side` value is `both` by default and may be `lo` or `hi` for one-sided cut boundaries. Ghost voxels do not carry independent mass; their surface ownership maps back to the corresponding real voxel. This is useful for making a finite slab or scan patch behave like an infinite wall during ISTHMUS marching cubes.

`voxel_write_history` writes the current in-memory history table immediately. Standalone inputs usually use `voxel_dump <id> <model> history ...`; the direct write command is mainly for DSMC-hosted scripts, where DSMC owns the loop and the core history should be flushed after the coupled loop completes.

9.4 Current Limits

- Only one voxel model is currently supported.

- Only one material is currently supported.
- Slab, centroid-filled sphere, and ISTHMUS-backed narrow 8-bit TIFF-stack

geometry are currently implemented.

- Ghost voxels currently support `boundary infinite`.

9.5 Planned Extensions

- CSV import.
- Periodic and open ghost voxel behavior.

10 `source` Command

The `source` command defines mass-loss data. It is available in standalone inputs and in DSMC-hosted inputs through the bridge.

10.1 Syntax

```
source <id> constant <value> units <units>
```

10.2 Example

```
source q1 constant 1.8 units kg/m2/s
```

10.3 Description

The current implementation supports a constant mass flux source. The value is interpreted as mass flux in `kg/m2/s` for the local slab ablation case.

In DSMC-hosted inputs, `source` is mainly useful for no-gas verification cases, debug probes, and prescribed ablation studies. Coupled DSMC chemistry cases usually use `surf_flux dsmc/reaction` or `surf_flux dsmc/surf` instead.

10.4 Current Limits

- Only `constant` sources are implemented.
- Units are recorded but not converted.

10.5 Planned Extensions

- Expressions.
- File or table sources.
- Surface-triangle sources.

11 `iac_timestep` Command

The `iac_timestep` command sets the IAC solid ablation time increment.

11.1 Syntax

```
iac_timestep <dt>
iac_timestep mass/courant <C> source <source-id>
```

11.2 Examples

```
iac_timestep 1.0e-6
iac_timestep mass/courant 0.5 source q1
```

11.3 Description

The explicit form sets `dt` directly.

The mass-Courant form chooses:

```
dt = C * rho * dx / source
```

For the current slab case, `C = 0.5` removes half of one voxel mass from each exposed column per step.

12 Loop Commands

The standalone runner supports a small DSMC/SPARTA-style loop subset.

12.1 Syntax

```
variable <name> loop <N>
variable <name> internal <value>
label <name>
next <name>
jump SELF <label>
iac_limit time <target>
iac_continue time <target> variable <name>
if "${name} > 0" then "jump SELF <label>"
```

12.2 Example

```
variable ablation_time equal 4.0e-3
variable keep internal 1
label ablate-loop
iac_limit time ${ablation_time}
voxel_ablate solid source q1 policy local face xlo delete yes
iac_run 1
iac_continue time ${ablation_time} variable keep
if "${keep} > 0" then "jump SELF ablate-loop"
```

12.3 Description

`variable <name> loop <N>` creates a loop counter.

`label <name>` marks a target location.

`next <name>` advances the loop counter. When the counter is exhausted, the next `jump` command is skipped.

`jump SELF <label>` jumps to a label in the current input.

`iac_limit time <target>` clips the current timestep if the next `iac_run` command would advance past the target. Put it inside the loop before `voxel_ablate` so the final mass-loss update and the recorded time end exactly at the target.

The current implementation supports only these compact loop patterns. It is meant to make standalone inputs resemble DSMC/SPARTA coupled scripts without reimplementing the full DSMC/SPARTA variable language.

13 `voxel_ablate` Command

The `voxel_ablate` command applies one timestep of mass loss to a voxel model.

13.1 Syntax

```
voxel_ablate <model> source <source-id> policy <policy> face <face> delete <yes|no>
voxel_ablate <model> surface <surface-id> policy <policy> delete <yes|no>
```

13.2 Example

```
voxel_ablate solid source q1 policy local face xlo delete yes
voxel_ablate solid source q1 policy local face yhi delete yes
voxel_ablate solid surface skin policy local delete yes
voxel_ablate solid surface skin policy carryover/normal delete yes
```

13.3 Description

The command applies one mass-loss update over the current timestep.

With `source`, `policy local` removes mass from the first active voxel in each slab column on the selected face. The required `face` argument accepts `xlo`, `xhi`, `ylo`, `yhi`, `zlo`, or `zhi`.

With `surface`, the command consumes mass already assigned to triangles by `surf_flux`, uses the ISTHMUS triangle-to-voxel ownership map, and subtracts that mass from the associated voxels.

`policy local` removes only the mass available in each mapped voxel and records any overshoot as dropped mass.

`policy carryover/normal` conserves overshoot by pushing excess mass loss from a depleted voxel into live 26-neighbor voxels along the inward normal direction. This policy is intended for closed ISTHMUS surfaces such as spheres.

Use `iac_run 1` after `voxel_ablate` to advance IAC solid time, record history, print stats, and write scheduled dumps:

```
voxel_ablate solid source q1 policy local face xlo delete yes
voxel_ablate solid source q1 policy local face zhi delete yes
iac_run 1
```

A surface-coupled loop is:

```
isthmus_surface skin voxels solid buffer 1 weighting no map yes
surf_flux skin source q1 select all
voxel_ablate solid surface skin policy carryover/normal delete yes
iac_run 1
```

`delete yes` is the default. When a voxel's mass is depleted, it is marked inactive and omitted from VTU dumps that use `select active`.

13.4 Current Limits

- Only one voxel model is currently supported.
- `policy carryover/normal` is currently implemented for surface-backed

ablation.

- The current source must be a constant mass flux.

14 `isthmus_surface` Command

The `isthmus_surface` command reconstructs a triangle surface from active voxels.

14.1 Syntax

```
isthmus_surface <surface-id> voxels <model> [buffer <N>] [resolution <R|A:B|voxel>] [weighting yes|no] [map yes|no] [
```

14.2 Example

```
isthmus_surface skin voxels solid buffer 1 weighting no map yes
isthmus_surface skin voxels solid buffer 1 weighting no map yes crop real
isthmus_surface skin voxels solid buffer 2 resolution 2:1 weighting no map yes
```

14.3 Description

The command sends the active voxels in `<model>` to ISTHMUS, runs marching cubes, stores the resulting triangle mesh, and optionally stores triangle-to-voxel ownership fractions.

`map yes` is required when the surface will be used for ablation, because `voxel_ablate <model> surface <surface-id>` needs those ownership fractions to send triangle mass loss back to voxels.

`buffer` adds empty voxel layers around the active voxel bounding box before marching cubes. This helps ISTHMUS generate a closed surface as the solid shrinks.

`resolution` controls the marching-cubes grid spacing relative to the voxel spacing. The default is `voxel`, equivalent to `1:1`, meaning one marching cell per voxel width. A value such as `2:1` makes the marching-cubes cell spacing twice the voxel spacing. Numeric values are accepted as shorthand, so `resolution 2` is equivalent to `resolution 2:1`. Larger values are coarser and usually need a larger `buffer` because each marching cell spans more voxel layers.

`crop real` removes triangles whose centroids are outside the real voxel domain after ghost voxels are added. Use it when ghost voxels are only a boundary condition for surface quality and flux should still be applied only on the real DSMC/voxel domain.

14.4 DSMC Coupling Note

This command is intentionally an explicit input-file command rather than a hidden fix callback. A future DSMC loop can call it after voxel mass changes and before refreshing the DSMC surface representation.

15 `surf_*` Commands

The `surf_*` command family works with triangle surfaces generated from voxels. These commands are deliberately separate from `voxel` commands because future DSMC-coupled runs will need to refresh surfaces, apply DSMC surface tallies, and then map that information back to voxels inside input-file loops.

15.1 Syntax

```
surf_flux <surface-id> source <source-id> select all
surf_flux <surface-id> source <source-id> select normal nx <x> ny <y> nz <z> [min-cos <c>]
surf_flux <surface-id> source <source-id> select voxels voxels <model>
surf_flux <surface-id> kinetic/theory pressure <p> temperature <T> \
  mole-fraction <x> molecular-mass <kg> reaction-prob <alpha> \
  solid-mass-per-hit <kg> select <selector>
surf_flux <surface-id> dsmc/surf fix <fix-id> quantity incident-number-flux \
  [reaction <path> | solid-mass-per-hit <kg>] \
  [reaction-prob <p>] [ablation-dt <dt> | mass-courant <C>]
surf_flux <surface-id> dsmc/surf fix <fix-id> column <N> \
  [reaction <path> | solid-mass-per-hit <kg>] \
  [reaction-prob <p>] [ablation-dt <dt> | mass-courant <C>]
surf_flux <surface-id> dsmc/reaction fix <fix-id> column <N> \
  sample-steps <Nstep> reaction <path> \
  [ablation-dt <dt> | mass-courant <C>] [time-scale <S>] \
  [select all | select normal nx <x> ny <y> nz <z> [min-cos <c>]]
surf_measure_flux <surface-id> dsmc/reaction fix <fix-id> column <N> \
  sample-steps <Nstep> expected kinetic/theory number-density <n> \
  mole-fraction <x> temperature <T> molecular-mass <kg> \
  [reaction <path>]

surf_dump <dump-id> <surface-id> vtp <N> <path>
surf_dump off

# DSMC bridge only:
surf_write_vtp <path> <surface-id> [fix <fix-id> fields <name1> ...]
```

15.2 Examples

```
surf_flux skin source q1 select all
surf_flux skin source q1 select normal nx 1.0 ny 0.0 nz 0.0 min-cos 0.5
surf_flux skin source q1 select voxels voxels solid
surf_flux skin kinetic/theory pressure 50.0 temperature 5000.0 \
  mole-fraction 0.21 molecular-mass 5.313e-26 reaction-prob 1.0 \
  solid-mass-per-hit 1.3011869411625376e-23 select all
surf_flux skin dsmc/surf fix sflux mass-courant 0.25
surf_flux skin dsmc/reaction fix rco column 1 sample-steps 20 \
  reaction carbon-co.surf mass-courant 0.1666666667
surf_flux skin dsmc/reaction fix rco column 1 sample-steps 100 \
  reaction carbon-co.surf mass-courant 1.0 \
  select normal nx 1.0 ny 0.0 nz 0.0 min-cos 0.5
surf_measure_flux skin dsmc/reaction fix rco column 1 sample-steps 1 \
  expected kinetic/theory number-density 7.244e23 mole-fraction 0.21 \
  temperature 5000.0 molecular-mass 5.31352e-26 reaction carbon-co.surf

surf_dump skin skin vtp 10 output/sphere/surface_*.vtp
surf_dump off

surf_write_vtp skin output/surface-*.vtp \
  fix sqty fields collision-count incident-number-flux pressure
```

15.3 Description

`surf_flux` assigns one timestep of mass loss to selected triangles on an existing surface. Source and kinetic-theory modes require an explicit `select all`, `select normal`, or `select voxels` clause. With `source`, the source is a constant mass flux in `kg/m2/s`. The requested mass on each selected triangle is:

```
mass = flux * triangle-area * timestep
```

With `kinetic/theory`, the command computes the mass flux from ideal-gas impingement theory for one reactive species:

```
Gamma = mole-fraction * pressure/(kB*temperature)
        * sqrt(kB*temperature/(2*pi*molecular-mass))
flux = reaction-prob * solid-mass-per-hit * Gamma
```

This is intended as the first DSMC-coupled verification source: DSMC can run a spatially uniform, stationary hot gas domain while this command applies the corresponding continuum kinetic-theory flux to the current ISTHMUS triangles.

With `dsmc/surf`, the command reads per-surface data from a DSMC `fix ave/surf` instance. The preferred pattern is to average only the incident number flux:

```
compute sflux surf all gas nflux_incident
fix sflux ave/surf all 1 100 100 c_sflux[*] ave one
```

Then either select it by name with `quantity incident-number-flux`, or omit the quantity because it is the default for `dsmc/surf`. The bridge converts the selected number flux to solid mass flux with:

```
flux = number-flux * reaction-probability * solid-mass-per-hit
```

If neither `reaction` nor `solid-mass-per-hit` is supplied, IAC infers one solid formula unit per incident hit from the current voxel material. For a material defined as `formula C molar-mass 0.0120107`, this compact default consumes one carbon atom per counted incident O2 molecule. It is useful for kinetic-theory collision-flux examples where DSMC is not actually running surface chemistry.

When `reaction <path>` is used, IAC reads the same SPARTA `surf_react prob` file used by DSMC. For a file such as:

```
O2 --> CO + CO
D S 1.0
```

and a material defined as `formula C molar-mass 0.0120107`, IAC infers a reaction probability of `1.0` and a solid mass per hit equal to two carbon atoms. The raw `solid-mass-per-hit` and `reaction-prob` arguments are retained as overrides for cases whose solid consumption cannot be inferred from the material formula or gas-side reaction formula.

This is the first true DSMC-coupled source path. DSMC owns the gas domain, surface collisions, surface averaging, loops, and remapping commands; this library owns the voxel mass ledger, ISTHMUS surface ownership map, and ablation. The current bridge implementation supports this source on one MPI rank while the command and data path are kept compatible with later distributed storage.

With `dsmc/reaction`, the command reads per-surface reaction counts from a DSMC `fix ave/surf`, usually one that averages `compute react/surf`. The selected column is interpreted as an averaged number of simulation-particle reactions per surface element per sampled timestep. The bridge converts it to a real solid mass rate over the sampled DSMC window:

```
mass = reaction-count * sample-steps * fnum * solid-mass-per-reaction
mflux = mass / (triangle-area * sample-steps * DSMC-timestep)
```

The solid timestep is then chosen by `ablation-dt`, `mass-courant`, or the sampled DSMC time when neither is specified. The requested triangle mass is `mflux * triangle-area * solid-timestep`. This is the preferred path for chemically reacting DSMC ablation because SPARTA owns the surface reaction probability, species conversion, and reaction tallies, while IAC owns the solid timestep and voxel mass ledger. Prefer `reaction <path>` so the solid mass per reaction is inferred from the same reaction file; use `solid-mass-per-reaction` only as an explicit override.

`surf_measure_flux` reads the same kind of DSMC reaction count data but does not apply mass loss. It sums the sampled reactions over the installed surface, converts them to a number flux with:

```
rflux = sum(reaction-count) * fnum / (area * DSMC-timestep)
```

and stores scalar diagnostics that can be checked later with `iac_verify`. The `expected kinetic/theory` arguments compute the ideal-gas one-way impingement flux for a reactive species:

```
rflux-exact =
  reaction-prob * mole-fraction * number-density
  * sqrt(kB*temperature/(2*pi*molecular-mass))
```

The command also stores `nreact` and `nreact-exact`, which are often the clearest DSMC sampling diagnostics. If `reaction` or `solid-mass-per-reaction` is supplied, it additionally stores `rmflux` and `rmflux-exact` in kg/m2/s by multiplying the number flux by the inferred or supplied solid mass consumed per reaction.

This command is the first DSMC-hosted regression hook for the coupled path: it tests geometry generation, surface installation, DSMC surface chemistry tallies, core diagnostics, and input-file verification without introducing voxel deletion or remeshing.

DSMC-coupled sources assign triangle mass loss and set the current IAC ablation timestep; they do not themselves advance the solid clock. Use `voxel_ablate` to map the pending triangle mass to voxels, then `iac_run 1` to record the solid step, history, dumps, and stats. By default, the bridge uses the elapsed DSMC time since the previous coupling update as the ablation time. The optional `ablation-dt` argument overrides that physical ablation time. For `dsmc/reaction`, `time-scale` scales the sampled DSMC window before the reaction rate is formed. In normal coupled runs, prefer `mass-courant` so the current local surface flux chooses a stable solid timestep.

For `dsmc/reaction`, an optional `select normal` clause restricts which reaction tallies are mapped back into voxel mass loss. SPARTA can still collide and react with the full installed surface; this selector only controls the solid ablation update. This is useful when a full watertight surface is needed for particles, but only one exposed face should recess in the cutout model.

The optional `mass-courant` argument chooses the ablation timestep from the largest current local voxel mass-removal rate. For surface fluxes, triangle mass rates are first mapped to active, non-fixed voxels through the ISTHMUS ownership fractions. The limiting rate is then converted to an equivalent voxel-face flux:

```
equivalent-face-flux = max(mvox-rate) / voxel-size^2
dt = C * mvox / max(mvox-rate)
```

This is the same nondimensional definition as ‘iac_timestep mass/courant‘: it is the mass that the local flux would remove through one voxel face during the timestep divided by one voxel mass. Thus `mass-courant 0.1666666667` is the conservative value `1/6`; even if all six faces of a voxel saw that same limiting flux, the update cannot remove more than one voxel mass before carryover/deletion handles the remainder. `ablation-dt` and `mass-courant` are mutually exclusive.

The mass is stored on the triangle until a later command consumes it:

```
voxel_ablate solid surface skin policy local delete yes
```

Selectors:

- `select all` applies flux to every triangle.
- `select normal` applies flux only when the triangle normal has

`dot(normal, direction) >= min-cos`.

- `select voxels` applies flux to triangles that have ISTHMUS ownership

entries for the named voxel model. This is a forward-compatible hook for voxel groups and material subsets.

`surf_dump` writes VTP triangle files on scheduled run steps. The VTP cell data includes `area`, `mreq`, `mreq-last`, `mflux`, `mflux-last`, and `selected`. The `last-*` fields are useful for visual inspection because `mreq` is cleared after `voxel_ablate` consumes it. `selected = 1` marks triangles that received flux during the most recent surface-flux command.

Inside DSMC, scheduled `surf_dump` output is written when bridge commands advance the core step. Define the dump before the first surface generation so the active core model is initialized with the desired schedule.

`surf_dump off` clears surface dumps that were already defined. Regression wrappers can use it after including a visual example input.

Inside DSMC, `surf_write_vtp` writes a one-shot VTP snapshot immediately. This mirrors `voxel_write_vtu`: DSMC owns the run loop, and the input script can write the current ISTHMUS surface after each `isthmus_surface` regeneration. If the path does not contain `*`, the current core step number is inserted before the file extension.

The optional `fix <fix-id> fields <name1> ...` form reads per-surface values from a DSMC `fix ave/surf` and appends them as VTP triangle cell fields. The number of field names must match the number of per-surface columns exposed by the fix. This is the preferred visualization path for coupled DSMC runs because the ISTHMUS triangle geometry and DSMC surface quantities are written into the same VTP file.

15.4 Current Limits

- Only VTP surface dumps are implemented.
- Constant source flux and single-species kinetic-theory flux are implemented.
- DSMC `fix ave/surf` flux ingestion currently supports one MPI rank.
- `select voxels` currently distinguishes ownership presence for the current

voxel model, not separate voxel groups.

16 iac_stats and iac_stats_style Commands

The iac_stats and iac_stats_style commands control IAC console output.

16.1 Syntax

```
iac_stats <N>
iac_stats_style <column> <column> ...
iac_spa_stats
```

16.2 Example

```
iac_stats 1
iac_stats_style step time nvox ndel mass mf vf front
```

16.3 Description

iac_stats N prints every N IAC steps and always prints step 0 and the final step.

iac_stats_style selects columns from tracked history quantities. The standalone runner prints a title/configuration block first, then a fixed-width iac_stats table.

16.4 Current Columns

```
step
time
nvox
ndel
mass
mf
vf
rad
front
mreq
mapp
mdrop
```

16.5 DSMC/SPARTA Note

In DSMC-hosted runs, use iac_stats and iac_stats_style for the core's own ablation table:

```
iac_stats 1
iac_stats_style step time nvox ndel mass mf rad
```

These commands do not change DSMC's native stats or stats_style settings. For coupled gas/solid runs, DSMC can continue owning particle/collision statistics while iac_stats prints the voxel mass ledger after IAC ablation steps.

iac_spa_stats is a DSMC-hosted helper for coupled logs. Place it after a native DSMC run and before the IAC ablation update:

```
stats_style     step np nscoll nscheck nsreact
stats           100000
run             100 post no
```

```
iac_spa_stats
voxel_ablate    solid surface skin policy carryover/normal delete yes
iac_run         1
```

On the first call, the bridge prints an `[IAC]` title/configuration block before the first `[SPA]` table. The block summarizes the voxel model, DSMC box, boundary conditions, timestep, particle weight, species/mixtures, surface collision/reaction models, and registered `grid_write_vtu` fluid outputs.

Each call then captures SPARTA's current native `stats_style` header and row, formats them as a fixed-width `[SPA]` table, and prints them at the gas-to-solid switch point. It does not modify the user's native `stats_style` command.

17 `voxel_dump` Command

The `voxel_dump` command writes voxel-model output.

Triangle surface dumps are configured with `surf_dump`.

17.1 Syntax

```
voxel_dump <id> <model> history <N> <path>
voxel_dump <id> <model> vtu <N> <path> [select all|active|ghosted|ghosts] [scalar <field>]
voxel_dump off

# Also available in the DSMC bridge:
voxel_dump <id> <model> history <N> <path>
voxel_dump <id> <model> vtu <N> <path> [select all|active|ghosted|ghosts] [scalar <field>]
voxel_write_vtu <model> <path> [select all|active|ghosted|ghosts] [scalar <field>]
```

17.2 Examples

```
voxel_dump hist solid history 1 output/slab-direct-ablation/history.csv
voxel_dump vox solid vtu 1 output/slab-direct-ablation/voxels_*.vtu select active scalar mf
voxel_dump off

voxel_write_vtu solid output/voxels-*.vtu select active scalar mf
```

17.3 Description

The `history` style writes one CSV summary after the run.

`voxel_dump off` clears voxel dumps that were already defined. This is useful in regression wrappers that include a visual example input but should avoid writing output files during CTest.

The `vtu` style writes VTK unstructured-grid files during the run. A dump is written at step 0, every `N` steps, and at the final step. If the path contains `*`, that character is replaced by a zero-padded step number:

```
output/slab-direct-ablation/voxels_000004.vtu
```

If the path does not contain `*`, the step number is inserted before the file extension.

The `select` option controls which voxels are written:

- `select all` writes every voxel, including inactive/deleted voxels.
- `select active` writes only active voxels.
- `select ghosted` writes active real voxels plus the ghost-image voxels used

for ISTHMUS surface generation. Ghost images are marked with cell data `ghost = 1` and do not contribute to mass totals.

- `select ghosts` writes only ghost-image voxels. This is the preferred visual

mode when an example needs to show boundary ghosts separately from the real solid.

The default is `select active`, so depleted voxels disappear from VTU output when the ablation fix uses `delete yes`.

The `scalar` option sets the active VTK cell scalar. This is the field ParaView should choose first when the file opens. Supported values are:

```
mf
mass
active
fixed
ghost
id
ix
iy
iz
```

The default is `scalar mf`.

Inside DSMC, scheduled `voxel_dump` output is written whenever a bridge command advances the core step, for example after `voxel_ablate`. This is useful for no-gas verification and coupled loops that use explicit DSMC input-file commands for ablation updates.

`voxel_write_vtu` writes a one-shot VTU snapshot immediately. This is useful in coupled loops where the input script decides exactly when a voxel snapshot should be written. It uses the same `select` and `scalar` options as scheduled `voxel_dump ... vtu` output. If the path does not contain `*`, the current core step number is inserted before the file extension.

17.4 History Columns

```
step,time,nvox,ndel,mass,mf,
vf,front,rad,mreq,mapp,
mdrop
```

17.5 VTU Cell Data

The VTU dump writes hexahedral voxel cells with:

```
mf
mass
id
ix
iy
iz
active
fixed
ghost
```

Use `active` for thresholding deleted voxels and `mf` or `mass` for coloring ablation progress in ParaView. Use `ghost` to hide or recolor mirror voxels that were emitted for boundary inspection.

17.6 Planned Extensions

- Restart dumps.
- Material/field output.

18 `iac_verify` Command

The `iac_verify` command compares tracked quantities to input-file exact/reference expressions.

18.1 Syntax

```
iac_verify <quantity> exact "<expression>" tolerance <value> [absolute|percent] norm <final|max|rms>
convergence <quantity> exact "<expression>" tolerance <value> [absolute|percent] norm <final|max|rms> vary <variable>
```

18.2 Examples

```
iac_verify mass exact "mass0 - q1*area*time" tolerance 0.01 percent norm max
iac_verify mf exact "1.0 - q1*time/(rho*length)" tolerance 0.01 percent norm max
iac_verify vf exact "1.0 - q1*time/(rho*length)" tolerance 0.01 percent norm final
iac_verify front exact "q1*time/rho" tolerance 0.01 percent norm final
iac_verify          rad exact "rad0 - q1*time/rho" tolerance 3.0 percent norm final
convergence         rad exact "rad0 - q1*time/rho" tolerance 100.0 percent norm final vary resolution 5 10 20 vary st
```

18.3 Description

The command evaluates an exact expression against the selected tracked quantity. The executable exits with a nonzero status if the error exceeds the tolerance. This makes verification cases natural CTest tests.

Regression tests generally use percent tolerances. Add `percent` after the tolerance value to check:

```
100 * abs(actual - exact) / abs(exact)
```

Percent error is undefined when the exact value is zero unless the actual value is also zero.

Use `absolute` or omit the mode when an absolute tolerance is more appropriate.

18.4 Convergence

`convergence` reruns the same input file with variable overrides and checks the error trend. Variables are defined with:

```
variable resolution equal 20
variable steps equal 4
```

and referenced with `${resolution}` or `${steps}` elsewhere in the input.

Each `vary` clause must have the same number of values. The first `vary` variable is treated as the refinement variable for apparent-order calculation. For example, `vary resolution 5 10 20 vary steps 1 2 4` runs three cases:

```
resolution=5,  steps=1
resolution=10, steps=2
resolution=20, steps=4
```

The convergence check requires monotone error reduction by default and checks that the apparent order from the first to last case is inside the requested `order <min> <max>` band.

18.5 Norms

`final` checks the last history row.

`max` checks the maximum absolute error over all recorded rows.

`rms` checks the root-mean-square error over all recorded rows.

18.6 Expression Variables

```
time
t
step
dt
rho
density
dx
nx
ny
nz
length
area
rad0
rad
mass0
mvox
nvox0
nvox
ndel
mass
mf
vf
front
q1
source:q1
```

19 `iac_run` Command

The `iac_run` command advances IAC solid time/history/dumps/stats. It is separate from DSMC/SPARTA's native `run` command, which advances particles and gas collisions.

19.1 Syntax

```
iac_run <steps>
iac_run duration <time>
```

19.2 Examples

```
iac_run 8
iac_run duration 0.02
```

19.3 Description

`iac_run <steps>` advances a fixed number of solid ablation steps.

`iac_run duration <time>` advances for a requested physical duration using the current timestep. If the final step would overshoot, the model clips that step so the recorded time ends exactly at the requested duration.

For explicit ablation loops, use `iac_limit time <target>` before `voxel_ablate`, then use `iac_continue` with DSMC-style 'if ... then "jump SELF <label>" `after` iac_run 1'.

20 Direct Slab Ablation

This example removes exactly half of one voxel mass from each exposed slab column per step.

Input file:

```
units si

voxel_material carbon density 1800.0
voxel_create solid slab nx 8 ny 4 nz 4 dx 1.0e-6 material carbon

source q1 constant 1.8 units kg/m2/s
iac_timestep mass/courant 0.5 source q1
variable ablation_time equal 4.0e-3
variable keep internal 1

iac_stats 1
iac_stats_style step time nvox ndel mass mf vf front

voxel_dump hist solid history 1 output/slab-direct-ablation/history.csv
voxel_dump vox solid vtu 1 output/slab-direct-ablation/voxels_*.vtu select active scalar mf

label ablate-loop
iac_limit time ${ablation_time}
voxel_ablate solid source q1 policy local face xlo delete yes
iac_run 1
iac_continue time ${ablation_time} variable keep
if "${keep} > 0" then "jump SELF ablate-loop"
```

Run it:

```
./build/ia-core -in examples/slab-direct-ablation/in.slab-direct-ablation
```

The example writes:

```
output/slab-direct-ablation/history.csv
output/slab-direct-ablation/voxels_000000.vtu
output/slab-direct-ablation/voxels_000001.vtu
...
output/slab-direct-ablation/voxels_000008.vtu
```

The VTU dump uses `select active`, so voxels removed by `delete yes` disappear from the visualized voxel mesh. It also uses `scalar mf`, so ParaView should open the files with `mf` as the active cell field.

The test form is:

```
include ../../../examples/slab-direct-ablation/in.slab-direct-ablation

iac_verify mass exact "mass0 - q1*area*time" tolerance 0.01 percent norm max
iac_verify mf exact "1.0 - q1*time/(rho*length)" tolerance 0.01 percent norm max
iac_verify front exact "q1*time/rho" tolerance 0.01 percent norm final
```

Run the test:

```
ctest --test-dir build --output-on-failure
```

Print the stats table during the test:

```
cmake --build build --target check-verbose
```

Build the visual verification report:

```
cmake --build build --target test-report
```

The combined report is written to:

```
build/output/test-report.pdf
```

To report only this test, run:

```
python3 tools/run-test-report.py slab-direct-command-verification \
  --out build/output/slab-direct-report.tex
```

21 ISTHMUS Slab Ablation

These examples ablate an 8 by 4 by 4 slab from the `xlo` ISTHMUS surface.

The finite-patch case intentionally exposes lateral edges:

```
examples/slab-isthmus-ablation/in.slab-isthmus-finite
```

It runs:

```
isthmus_surface skin voxels solid buffer 1 weighting no map yes
surf_flux skin source q1 select normal nx -1.0 ny 0.0 nz 0.0 min-cos 0.5
voxel_ablate solid surface skin policy local delete yes
```

The ghost case adds y/z infinite-wall ghost voxels before surface generation:

```
voxel_ghost solid axis y boundary infinite layers 1
voxel_ghost solid axis z boundary infinite layers 1
isthmus_surface skin voxels solid buffer 1 weighting no map yes
```

Ghost voxels are surface-construction images only. They do not carry separate mass, and triangle ownership maps back to the corresponding real voxels. This removes finite-patch edge effects while keeping the ablation update on the real voxel domain.

Run either case:

```
build/ia-core -in examples/slab-isthmus-ablation/in.slab-isthmus-finite
build/ia-core -in examples/slab-isthmus-ablation/in.slab-isthmus-ghost
```

The regression wrappers are:

```
tests/inputs/slab-isthmus-ablation/in.slab-isthmus-finite-surface.verify
tests/inputs/slab-isthmus-ablation/in.slab-isthmus-ghost-wall.verify
```

The finite-patch wrapper uses broad tolerances because the exposed edges are expected to perturb the one-dimensional exact recession. The ghost wrapper uses tight tolerances because the infinite-wall construction should recover the one-dimensional slab solution.

22 Sphere ISTHMUS Ablation

This example reconstructs a triangle surface from a voxel sphere with ISTHMUS, applies a constant mass flux to all surface triangles, maps the triangle mass loss back to voxels, and deletes depleted voxels.

The current case uses a 1 mm diameter sphere with `resolution 20`, meaning 20 voxels across the diameter. It uses `iac_timestep mass/courant 0.5 source q1`, which gives `dt = 0.05625 s` for the current material, grid spacing, and flux. The four-step test therefore runs to `0.225 s`.

22.1 Inputs

```
examples/sphere-isthmus-ablation/in.sphere-isthmus-local
examples/sphere-isthmus-ablation/in.sphere-isthmus-normal
```

The core loop is:

```
isthmus_surface skin voxels solid buffer 1 weighting no map yes
surf_flux skin source q1 select all
voxel_ablate solid surface skin policy carryover/normal delete yes
iac_run 1
```

This loop is intentionally command-driven. A future DSMC run can place DSMC convergence, flux mapping, voxel ablation, and surface regeneration in the same style of input-file loop.

22.2 Run

```
build/ia-core -in examples/sphere-isthmus-ablation/in.sphere-isthmus-local
build/ia-core -in examples/sphere-isthmus-ablation/in.sphere-isthmus-normal
```

22.3 Outputs

```
output/sphere-isthmus-local/
output/sphere-isthmus-normal/
```

Open the VTU files in ParaView to inspect active voxel mass fraction. Open the VTP files to inspect surface triangle area and the last applied surface mass request.

22.4 Verification

The automated wrappers are:

```
tests/inputs/sphere-isthmus-ablation/in.sphere-isthmus-local-deletion.verify
tests/inputs/sphere-isthmus-ablation/in.sphere-isthmus-normal-carryover.verify
```

It compares the inferred continuum-equivalent radius (`rad`) and mass fraction against the analytical shrinking-sphere solution:

```
rad = rad0 - q1*time/rho
mf = (rad / rad0)^3
```

The local wrapper is the intentionally nonconservative reference path. It uses percent tolerances of `7.0 percent` for `rad` and `20.0 percent` for mass fraction at the final time.

The normal-carryover wrapper conserves mapped surface mass by pushing overshoot inward through live neighbor voxels. It uses tighter percent tolerances of `4.0 percent` for `rad` and `12.0 percent` for mass fraction, and it checks that final-step dropped mass is essentially zero.

23 TIFF Sphere Example

This example imports a synthetic multipage TIFF sphere through the ISTHMUS TIFF utility, reconstructs an ISTHMUS surface, applies a constant surface flux, and maps the mass loss back to voxels with normal-directed carryover.

The committed fixture is:

```
examples/tiff-sphere/sphere-24.tif
```

It is an uncompressed 8-bit grayscale TIFF stack with `24 x 24 x 24` samples. ISTHMUS's TIFF utility treats pixel value `1` as active solid and value `0` as void. The imported active bounding lattice reports as `23 x 23 x 23` for this fixture because the outermost TIFF samples remain void. The fixture is generated reproducibly with:

```
python3 tools/make-tiff-sphere.py --size 24 --radius 11.5 \
  --out examples/tiff-sphere/sphere-24.tif
```

23.1 Input

The import command is:

```
voxel_create solid tiff file examples/tiff-sphere/sphere-24.tif dx 5.0e-5 material carbon
```

The ablation loop is the same command-driven loop used by generated sphere cases:

```
iac_limit time ${ablation_time}
isthmus_surface skin voxels solid buffer 1 weighting no map yes
surf_flux skin source q1 select all
voxel_ablate solid surface skin policy carryover/normal delete yes
iac_run 1
iac_continue time ${ablation_time} variable keep
if "${keep} > 0" then "jump SELF ablate-loop"
```

23.2 Run

```
make standalone
./build/ia-core -in examples/tiff-sphere/in.tiff-sphere-normal
```

The example writes:

```
output/tiff-sphere-normal/history.csv
output/tiff-sphere-normal/voxels_*.vtu
output/tiff-sphere-normal/surface_*.vtp
```

Open the VTU and VTP files in ParaView to inspect the imported voxel sphere and the reconstructed surface as the sphere recesses.

23.3 Verification

The automated wrapper is:

```
tests/inputs/tiff-sphere/in.tiff-sphere-normal-carryover.verify
```

It disables visual dumps and compares final mass fraction and voxelized volume fraction to the continuum shrinking-sphere solution. Because the sphere is imported as arbitrary TIFF geometry, the exact expression computes an equivalent initial radius from the imported initial mass:

```
R0 = (3*mass0/(4*pi*rho))^(1/3)
mf = ((R0 - q1*time/rho) / R0)^3
```

This case verifies two things at once: the TIFF import path uses `isthmus::utilities::load_active_voxels_from_tiff`, and the imported voxel state can drive the same ISTHMUS surface-flux-to-voxel ablation loop as generated geometries.

24 DSMC Sphere Kinetic Example

This example runs from this repository using the private DSMC/IAC executable built by the overlay target. It is the simplest coupled DSMC recession case: SPARTA owns particles, surface collisions, `compute surf`, `fix ave/surf`, loops, `remove_surf`, and `surf_modify`; IAC owns voxel state, ISTHMUS surface generation, collision-flux-to-triangle conversion, normal-carryover ablation, and regenerated surface geometry.

Run it directly with:

```
make mpi

cd examples/dsmc-sphere-kinetic
../../build-dsmc/bin/dsmc-iac \
  -screen none -log output/log.sparta -in in.dsmc-sphere-kinetic
```

To run the DSMC-enabled CTest suite:

```
make test-dsmc
```

The normal standalone build does not include DSMC tests. A DSMC-enabled build runs the standalone tests plus DSMC-hosted bridge tests.

24.1 Collision-Flux Loop

The case uses pure O2 gas in a periodic 3 mm cube. Gas chemistry and gas-gas collisions are disabled on purpose. This makes the comparison a direct test of ideal-gas thermal impingement, DSMC surface-collision tallying, triangle-to- voxel mapping, normal carryover, and repeated ISTHMUS remeshing.

The gas and wall are both 5000 K:

```
boundary            p p p
species             air.species O2
mixture             gas O2 frac 1.0
mixture             gas temp 5000.0 vstream 0.0 0.0 0.0
surf_collide        1 diffuse 5000.0 1.0
```

SPARTA samples the incident number flux on each surface triangle:

```
compute             sflux surf all gas nflux_incident
fix                 sflux ave/surf all 1 100 100 c_sflux[*] ave one
run                 100 post no
```

IAC reads the incident number flux and converts it to carbon mass flux:

```
surf_flux skin dsmc/surf fix sflux mass-courant 0.1666666667
iac_limit time ${ablation_time}
voxel_ablate solid surface skin policy carryover/normal delete yes
```

Because chemistry is intentionally disabled here, the command uses the compact `dsmc/surf` default: the incident flux is treated as a perfectly reactive collision flux and one carbon formula unit is consumed per counted O2 hit. With `voxel_material carbon ...  molar-mass 0.0120107 formula C`, that means one carbon atom per hit. This is deliberately simpler than the reacting `surf_react` path; if the physical interpretation should be `O2 -> CO + CO`, the analytical comparison differs by a factor of two in solid mass consumed per incident O2 molecule. The DSMC-measured incident flux is still applied triangle-by-triangle, then mapped back to voxels through the ISTHMUS ownership fractions.

After each ablation update, native DSMC commands clear the old collision surface and install the regenerated ISTHMUS surface directly from memory:

```
remove_surf all
isthmus_surface skin voxels solid buffer 1 weighting no map yes
surf_install skin particle none type 1
surf_modify all collide 1
```

The committed example uses a 10-voxel sphere, `C_m = 1/6`, and a named `ablation_time` target. `iac_limit time` clips the last solid timestep so the history ends at exactly that physical ablation time while exercising the full DSMC-to-ISTHMUS-to-voxel loop. It is a workflow example. The smaller committed regression version is `dsmc-sphere-kinetic-convergence`, which runs generated 4-, 6-, and 8-voxel inputs and checks grid convergence in the default DSMC CTest suite.

24.2 Analytical Comparison

For pure O2, the one-way thermal impingement flux is:

```
Gamma = n * sqrt(kB*T/(2*pi*m-O2))
j = Gamma * reaction-probability * solid-mass-per-hit
R(t) = R0 - j*t/rho-solid
mf = (R/R0)^3
```

The example writes:

```
examples/dsmc-sphere-kinetic/output/history.csv
examples/dsmc-sphere-kinetic/output/voxels_*.vtu
examples/dsmc-sphere-kinetic/output/surface_*.vtp
```

The VTU files show active voxels with `mf` as the selected cell field, including the initial voxel state. The VTP files show regenerated ISTHMUS triangle surfaces after solid ablation steps.

Check its final mass fraction against the continuum solution with:

```
python3 ../../tools/check-dsmc-kinetic.py output/history.csv \
  --number-density 7.244e23 \
  --temperature 5000 \
  --molecular-mass 5.31352e-26 \
  --solid-density 1800 \
  --solid-molar-mass 0.0120107 \
  --solid-atoms-per-hit 1 \
  --rad0 5e-4
```

24.3 Chemistry Note

The repository still includes a one-step DSMC chemistry flux verification in:

```
tests/inputs/dsmc-sphere-flux/in.dsmc-sphere-flux.verify
```

That test keeps the `dsmc/reaction` bridge path covered. The main coupled sphere recession example intentionally uses collision flux instead, because it isolates the kinetic-theory comparison from chemistry, composition depletion, and heat-release modeling.

25 Pregen TIFF Carbon Recession

This example folder contains two small rough-carbon recession cases built from the same deterministic TIFF stack:

```
examples/pregen-tiff-carbon-recession/
  generate-sample.sh
  in.constant-flux
  in.dsmc-co
  air.species
  air.vss
  carbon-co.surf
```

The generated sample is mostly solid carbon with shallow random pits at the top surface. It is intentionally synthetic and reproducible, so the repository does not need to carry a binary scan file.

25.1 Generate the TIFF

From the repository root:

```
examples/pregen-tiff-carbon-recession/generate-sample.sh
```

This writes:

```
examples/pregen-tiff-carbon-recession/carbon-sample.tif
```

The default sample is `24 x 24 x 10` voxels and contains `4650 / 5760` active voxels. The generated TIFF is ignored by Git.

25.2 Constant-Flux Case

Run the standalone case from the example folder:

```
make standalone
cd examples/pregen-tiff-carbon-recession
./generate-sample.sh
./clean-output.sh
../../build/ia-core -in in.constant-flux
```

The case imports the TIFF, places infinite-wall ghost voxels on the four lateral sides, supports the `x-lo` side with one-sided ghost voxels, reconstructs the exposed pitted `x-hi` surface with ISTHMUS, applies a constant mass flux along `+x`, and ablates with normal-directed carryover:

```
voxel_ghost         solid axis y boundary infinite layers 1
voxel_ghost         solid axis z boundary infinite layers 1
voxel_ghost         solid axis x side lo boundary infinite layers 1
surf_flux           skin source qtop select normal nx 1.0 ny 0.0 nz 0.0 min-cos 0.5
voxel_ablate        solid surface skin policy carryover/normal delete yes
```

The example writes real voxels and ghost voxels as separate VTU series. The ghost files use `select ghosts`, so ParaView can show the mirror-image ghost voxels used by ISTHMUS without mixing them into the real solid. The surface VTP files dump the full closed real-plus-ghost ISTHMUS surface. They include `mflux-last` and `selected` cell fields, so the full surface can be colored by applied flux while unselected triangles remain visible with zero flux.

With the committed parameters, the case recesses to about 0.70 remaining mass fraction and 0.75 remaining voxelized volume fraction. This makes it a standalone sister case to the DSMC chemistry example below.

It writes visual output under the example folder:

```
output/pregen-tiff-carbon-recession/constant-history.csv
output/pregen-tiff-carbon-recession/constant-voxels_*.vtu
output/pregen-tiff-carbon-recession/constant-ghosts_*.vtu
output/pregen-tiff-carbon-recession/constant-surface_*.vtp
```

25.3 DSMC CO-Chemistry Case

Run the coupled DSMC case from its folder so the local species and reaction files resolve naturally:

```
make mpi
cd examples/pregen-tiff-carbon-recession
./generate-sample.sh
./clean-output.sh
../../build-dsmc/bin/dsmc-iac -log output/pregen-tiff-carbon-recession/log.sparta -in in.dsmc-co
```

This case installs the full watertight ISTHMUS surface into DSMC/SPARTA, creates a hot pure-O2 stream at 8000 K entering from the xhi face toward the sample's pitted xhi surface, applies the SPARTA-style surface reaction

```
O2 --> CO + CO
D S 1.0
```

and maps compute react/surf reaction counts back to carbon voxels through the ISTHMUS triangle-to-voxel map. It is a compact workflow example, not a high-fidelity oxidation validation case; gas-gas collisions are intentionally omitted to keep it cheap.

The chemistry-driven flux is sampled from compute react/surf and the solid timestep is selected with mass-courant 1.0, so the example advances through several small voxel-ablation updates instead of consuming the full sampled reaction mass in one step. Each coupling update samples 100 DSMC steps before mapping reaction counts back to the voxels. The input uses SPARTA's run ... every 25 "iac_spa_stats" hook to print compact [SPA] gas-domain progress during the 100-step sample without spelling out a second input loop. The installed SPARTA surface is the full watertight ISTHMUS surface, but the ablation flux is restricted to triangles facing x+ with 'select normal nx 1.0 ny 0.0 nz 0.0 min-cos 0.5'. The committed input runs to 2.0e-7 s, reaches about 0.70 remaining mass fraction and 0.75 voxelized volume fraction, and stops before the remaining rough geometry reaches the current SPARTA watertight-check limit.

It writes visual output under:

```
output/pregen-tiff-carbon-recession/dsmc-history.csv
output/pregen-tiff-carbon-recession/dsmc-voxels_*.vtu
output/pregen-tiff-carbon-recession/dsmc-ghosts_*.vtu
output/pregen-tiff-carbon-recession/dsmc-surface_*.vtp
output/pregen-tiff-carbon-recession/fluid_*.vtu
```

The fluid_*.vtu files are written from a SPARTA fix ave/grid and include cell temperature, pressure, and O2/N2/CO mass fractions. The input uses grid_write_vtu once to write fluid_000000.vtu, then uses scheduled grid_dump output with index iac-step so later fluid files share the same ablation-step numbering as the voxel and surface files. It also writes dsmc-surface_000000.vtp before the coupling loop so ParaView can open the fluid, voxel, and surface series together from the same initial index.

The first compact output block is an [IAC] configuration summary covering the voxel model and the DSMC/SPARTA setup: domain bounds, boundary conditions, species/mixtures, surface chemistry, and fluid dump paths. After that, iac_spa_stats prints SPARTA gas statistics as a fixed-width [SPA] table during each gas sample, followed by IAC voxel ledger statistics with an [IAC] prefix after the solid update.

25.4 Regression Tests

The standalone constant-flux wrapper runs without visual dumps:

```
ctest --test-dir build -R pregen-tiff-carbon-recession-constant-flux-verification --output-on-failure
```

The DSMC chemistry wrapper runs the same generated sample through the DSMC/IAC executable:

```
ctest --test-dir build-dsmc -R pregen-tiff-carbon-recession-dsmc-co-verification --output-on-failure
```

26 Directory Layout

```
include/isthmus_ablation/
  Public C++ headers for the core library.

src/
  Core implementation, parser, expression evaluator, and standalone CLI.

examples/
  Human-readable standalone input examples, organized as <case>/in.<case>.

dsmc-bridge/
  DSMC command-style bridge sources. The dsmc CMake target symlinks these into
  build-dsmc/dsmc-overlay/src; they should not be copied into the DSMC checkout.

tests/inputs/
  Regression wrappers, organized as <case>/in.<case>.verify.

docs/
  Manual, command reference, design notes, and generated diagram/PDF artifacts.

tools/
  Local documentation and development utilities.

build-dsmc/dsmc-overlay/
  Generated private DSMC source overlay. This is disposable build output.
```

Generated files are ignored:

```
build/
build-dsmc/
output/
site/
```

The generated single-file manual is:

```
docs/isthmus-ablation-core-manual.pdf
```

27 Code Architecture

This page is a short map for developers reading the code for the first time.

27.1 Core Library

The standalone and DSMC-hosted paths both call the same core library.

| Path | Purpose | | — | — | | `include/isthmus_ablation/types.hpp` | Plain data structures for input configuration, voxel state, surfaces, history, and checks. | | `include/isthmus_ablation/model.hpp` | Public `iac::Model` API used by the standalone executable and DSMC bridge. | | `src/model.cpp` | Voxel geometry, material accounting, mass removal, ISTHMUS calls, dumps, diagnostics, and verification. | | `src/parser.cpp` | Standalone input parser. It accepts the same IAC command names used by the DSMC bridge. | | `src/expression.cpp` | Small expression evaluator for exact solutions and verification criteria. | | `src/main.cpp` | Standalone `ia-core` executable and convergence/report orchestration. |

27.2 DSMC Bridge

The DSMC bridge lives in `dsmc-bridge/`. The build creates a private DSMC overlay in `build-dsmc/dsmc-overlay/src` and symlinks these files there. The user's DSMC checkout is not edited.

| Path | Purpose | | — | — | | `dsmc-bridge/iacbridge.*` | Shared bridge state: one rank-zero core model, config, diagnostics, stats output, and MPI broadcasts. | | `dsmc-bridge/voxel.*` | `voxel_*` command wrappers. | | `dsmc-bridge/isthmus.*` | `isthmus_surface` command wrapper and surface broadcast. | | `dsmc-bridge/surface.*` | `surf_*` wrappers for surface install, flux ingestion, flux measurement, and VTP output. | | `dsmc-bridge/source.*` | `source` command wrapper. | | `dsmc-bridge/iac.*` | `iac_*` wrappers for solid timestep, run, stats, verification, and loop variables. |

27.3 Execution Model

Standalone execution parses the whole input into a `Config`, constructs a `Model`, then executes the command program inside `Model::run`.

DSMC execution is command-by-command. DSMC parses the input and calls bridge classes directly. Those classes update `IACBridge::config()`, create or mutate the rank-zero `Model`, and call the same core methods used by standalone mode.

The important boundary is:

```
DSMC owns gas time:      run, timestep, particles, collisions, surface tallies
IAC owns solid time:     iac_timestep, iac_limit, voxel_ablate, iac_run
ISTHMUS owns meshing:    isthmus_surface through the core surface adapter
```

27.4 Adding A Command

Prefer adding behavior in this order:

- Add or extend a plain data structure in `types.hpp`.
- Implement the core behavior in `Model`.
- Add standalone parsing in `src/parser.cpp`.
- Add the DSMC bridge wrapper only if the command must be callable inside DSMC.
- Add or update one example and one focused test.

- Update command docs and rebuild the manual PDF.

Commands should stay explicit. Avoid hidden advancement: commands that apply mass loss should not also advance time unless their name says so. Use `iac_run` for solid advancement.

28 Build And Link

This project has two build modes:

- standalone `ia-core`, which runs only this repository's input commands;
- DSMC/IAC overlay, which builds a private DSMC executable with IAC commands.

The overlay is the recommended user workflow. It does not copy files into the user's DSMC checkout and it does not edit DSMC source files.

28.1 What Is Built

The core library is:

```
libisthmus_ablation_core.a
```

When ISTHMUS is found, that library links against:

```
libisthmus_cpp.a
```

The standalone executable is:

```
build/ia-core
```

The DSMC/IAC executable is:

```
build-dsmc/bin/dsmc-iac
```

28.2 Root Discovery

CMake looks for DSMC and ISTHMUS in this order:

- Make variables passed through to CMake:

```
make mpi DSMC_ROOT=/path/to/dsmc ISTHMUS_ROOT=/path/to/isthmus
```

- Environment variables:

```
export DSMC_ROOT=/path/to/dsmc
export ISTHMUS_ROOT=/path/to/isthmus
```

- Common local paths:

```
../dsmc
~/dsmc
../isthmus/build
../isthmus/build-wsl
~/isthmus/build
~/isthmus/build-wsl
```

`ISTHMUS_ROOT` should point to an ISTHMUS checkout or install that contains a CMake package under `build/` or `build-wsl/`. Users can also set `ISTHMUS_CPP_DIR` or `isthmus_cpp_DIR` directly to the directory containing `isthmus_cppConfig.cmake`.

28.3 Recommended DSMC/IAC Build

One-time shell setup:

```
export DSMC_ROOT=$HOME/dsmc
export ISTHMUS_ROOT=$HOME/isthmus
```

Build and test:

```
make mpi
make test-dsmc
```

The top-level `Makefile` is the recommended user interface. It keeps the same machine-target style as DSMC/SPARTA while CMake still handles package discovery, test registration, generated fixtures, and overlay generation.

Common targets are:

```
make standalone
make mpi
make mac_mpi
make serial
make dsmc DSMC_MACHINE=<machine>
make check-mpi
```

28.4 University Of Kentucky MCC

The Morgan Compute Cluster can use its default GNU/OpenMPI stack. Load CMake and use the top-level make workflow:

```
module load cmake
cmake --version
```

Then set the external code roots and build normally:

```
export DSMC_ROOT=$HOME/TylerStoffel/dsmc
export ISTHMUS_ROOT=$HOME/TylerStoffel/isthmus

make mpi
make test-dsmc
```

The normal build/test path uses MCC's system Python 3.6 for small helper scripts. Documentation and report-generation targets may need additional LaTeX or Python packages, but they are not required for compiling and running the verification suite.

Run a coupled input:

```
build-dsmc/bin/dsmc-iac \
  -in examples/dsmc-sphere-kinetic/in.dsmc-sphere-kinetic
```

28.5 DSMC Machine Target

The default DSMC make target is:

```
mpi
```

Override it with:

```
make mac_mpi
```

or:

```
make dsmc DSMC_MACHINE=mac_mpi
```

The executable generated by DSMC inside the overlay is `spa_<machine>`, and this repository creates a stable launcher:

```
build-dsmc/bin/dsmc-iac
```

The launcher quiets native SPARTA screen output by default so coupled runs show the compact IAC bridge summaries in the terminal while full SPARTA output still goes to the log file. Add `-iac-dsmc-verbose` to restore native SPARTA screen output:

```
build-dsmc/bin/dsmc-iac --iac-dsmc-verbose -in examples/.../in.case
```

The launcher also accepts `-iac-color auto|on|off`. The default is `auto`, which colors compact `[IAC]` lines green and `[SPA]` lines blue only when stdout is a terminal. Log files are always written without color escape codes.

28.6 MPI Test Discovery

The DSMC overlay is built with the `mpi` DSMC machine target by default. CMake also looks for an MPI launcher such as `mpiexec` or `mpirun`. When one is available, every DSMC input-file CTest is registered once in serial and once under MPI. Python-driven orchestration tests are kept serial-only unless they explicitly add MPI support. Configure the rank count with:

```
make mpi IAC_DSMC_MPI_NP=4
```

To point at a specific launcher:

```
make mpi CMAKE_ARGS="-DIAC_MPIEXEC_EXECUTABLE=/path/to/mpiexec -DIAC_MPIEXEC_NUMPROC_FLAG=-n"
```

If no launcher is found, CMake warns and skips only the MPI tests. Disable MPI test registration explicitly with:

```
make mpi IAC_ENABLE_MPI_TESTS=OFF
```

28.7 DSMC MPI Runtime Model

The current DSMC bridge uses a rank-0 IAC ownership model. MPI rank 0 owns the voxel model, material ledger, ISTHMUS calls, history, diagnostics, and IAC file outputs. Other DSMC ranks still parse the same input commands, but they do not instantiate their own voxel model.

When `isthmus_surface` generates a surface, rank 0 converts the current voxel state to an ISTHMUS surface and broadcasts the public triangle geometry to all DSMC ranks. `surf_install` then partitions those triangles across DSMC ranks for normal SPARTA surface/grid/particle work. DSMC surface tallies are reduced across ranks before rank 0 maps the resulting triangle fluxes back into voxel mass.

Commands that need scalar IAC state for DSMC input control, such as `iac_continue` and `iac_set`, compute the value on rank 0 and broadcast it so all ranks make the same input-script decisions.

This is the first MPI level. It avoids duplicated voxel/ISTHMUS work and keeps the IAC state deterministic, but the voxel model itself is not distributed yet. Large voxel samples may eventually need distributed voxel ownership, distributed ISTHMUS calls, or restart-aware domain decomposition.

28.8 How The Overlay Works

The `dsmc` target creates:

```
build-dsmc/
  bin/
    dsmc-iac
  dsmc-overlay/
    README.md
    src/
      symlinks to DSMC src files
      symlinks to dsmc-bridge/*.cpp and *.h
      generated Makefile.package
      generated Makefile.package.settings
      Obj_<machine>/
      spa_<machine>
```

The script that creates this is:

```
tools/create-dsmc-overlay.py
```

It symlinks the user's DSMC `src` directory into the overlay, excluding build artifacts and generated style headers. It then symlinks these IAC bridge files:

```
dsmc-bridge/iac.cpp
dsmc-bridge/iac.h
dsmc-bridge/iacbridge.cpp
dsmc-bridge/iacbridge.h
dsmc-bridge/iacgrid.cpp
dsmc-bridge/iacgrid.h
dsmc-bridge/isthmus.cpp
dsmc-bridge/isthmus.h
dsmc-bridge/source.cpp
dsmc-bridge/source.h
dsmc-bridge/surface.cpp
dsmc-bridge/surface.h
dsmc-bridge/voxel.cpp
dsmc-bridge/voxel.h
```

The overlay also symlinks this repository's public include directory:

```
include/isthmus_ablation -> build-dsmc/dsmc-overlay/src/isthmus_ablation
```

This keeps bridge includes such as `#include "isthmus_ablation/model.hpp"` resolvable even during DSMC's dependency-generation rules, before the normal package include flags are applied consistently by every machine makefile.

DSMC's existing make system scans headers in the overlay and regenerates `style_command.h`. Because the bridge headers are present in the overlay, commands such as:

```
voxel_create ...
voxel_ablate ...
isthmus_surface ...
surf_flux ...
source ...
iac_run ...
```

become visible to DSMC without editing DSMC parser logic.

The overlay `Makefile.package` adds:

```
-I/path/to/isthmus-ablation-core/include
-I/path/to/isthmus/include
/path/to/isthmus-ablation-core/build-dsmc/libisthmus_ablation_core.a
/path/to/isthmus/build/libisthmus_cpp.a
```

or the equivalent paths discovered from the ISTHMUS CMake package.

28.9 Standalone Build

Standalone mode only needs this repository and, for ISTHMUS cases, ISTHMUS:

```
make standalone
make test-standalone
```

Run a standalone ISTHMUS example:

```
./build/ia-core -in examples/sphere-isthmus-ablation/in.sphere-isthmus-normal
```

If ISTHMUS is not found, CMake still builds direct voxel functionality, but ISTHMUS tests and examples are unavailable.

28.10 Debugging The Overlay

The overlay is disposable. To force a clean DSMC/IAC rebuild:

```
rm -rf build-dsmc/dsmc-overlay build-dsmc/bin/dsmc-iac
make mpi
```

To inspect the generated DSMC package settings:

```
cat build-dsmc/dsmc-overlay/src/Makefile.package
```

To bypass the overlay and run tests against an existing DSMC executable:

```
cmake -S . -B build-dsmc-external \
  -DIAC_DSMC_USE_OVERLAY=OFF \
  -DIAC_DSMC_EXECUTABLE=/path/to/spa_mac_mpi
cmake --build build-dsmc-external
ctest --test-dir build-dsmc-external --output-on-failure
```

That external path is mainly for debugging old builds. New users should prefer the overlay.

29 Testing

The clean full test path is the DSMC/IAC make wrapper. It builds the core library, standalone executable, private DSMC/IAC executable, and then runs the full CTest matrix:

```
make check-mpi
```

For a completely fresh rebuild:

```
make clean
make check-mpi
```

Inspect the registered tests without running them:

```
ctest --test-dir build-dsmc -N
```

The DSMC/IAC test suite includes:

- standalone `ia-core` regression tests;
- the same pure-IAC inputs hosted through `build-dsmc/bin/dsmc-iac`;
- MPI-hosted twins when CMake finds an MPI launcher;
- DSMC gas-domain checks for surface flux measurement, bridge behavior, and a

small particle-driven kinetic sphere recession convergence case.

For standalone-only development:

```
make check-standalone
```

For local environments where MPI launch is unavailable or blocked, run the DSMC-hosted non-MPI tests only:

```
make test-dsmc-serial
```

CTest captures output from passing tests by default. To see stats output while tests run:

```
make test-standalone CTEST_ARGS=--verbose
```

or:

```
ctest --test-dir build --output-on-failure --verbose
```

More detail on test categories, input-file conventions, and pass/fail criteria lives in `tests/README.md`.

29.1 Test Reports

The optional visual verification report is separate from the default test run:

```
make report
```

This runs the configured report cases, collects verification CSV files, and writes:

```
build/output/test-report.pdf
```

To select a subset directly:

```
python3 tools/run-test-report.py all
python3 tools/run-test-report.py 1-3
python3 tools/run-test-report.py slab-direct-command-verification
```

Keep slow exploratory plotting and convergence reports out of the default CTest suite until they are deterministic and fast enough to serve as regression tests.

The coupled DSMC kinetic recession regression is intentionally small:

```
ctest --test-dir build-dsmc -R dsmc-sphere-kinetic-convergence --output-on-failure
```

It runs three generated DSMC inputs at 4, 6, and 8 voxels across the sphere diameter and checks that the particle-sampled recession converges toward the ideal-gas kinetic-theory mf solution.

The rough carbon TIFF workflow checks can be run directly with:

```
ctest --test-dir build -R pregen-tiff-carbon-recession-constant-flux-verification --output-on-failure
ctest --test-dir build-dsmc -R pregen-tiff-carbon-recession-dsmc-co-verification --output-on-failure
```

The first uses a prescribed top-surface mass flux. The second uses DSMC surface chemistry (`O2 -> CO + CO`) to drive the same kind of voxel recession on the same generated rough TIFF fixture.

30 Test Reports

Verification reports are optional visual artifacts for inspecting regression cases. They are not part of the core physics model.

30.1 Command-Line Report Output

The executable can write verification data when the test runner asks for it:

```
build/ia-core -in tests/inputs/slab-direct-ablation/in.slab-direct-command.verify \
  -report-csv build/output/test-report-data/slab-direct-command-verification/report.csv
```

The CSV contains:

```
quantity,expression,step,time,actual,exact,error,tolerance,tolerance-mode,norm,pass
```

This data-first design is intentional. Future DSMC/SPARTA-coupled runs can emit the same verification/report CSV without depending on standalone plotting code or modifying the input file.

30.2 Report Cases

Reportable tests are listed in:

```
tests/report-cases.csv
```

Each row gives the report index, test name, input file, and optional requirements. Add future regression cases there when they should be available to the report runner.

```
index,name,input,requires,expected-fail
2,slab-direct-yhi-verification,tests/inputs/slab-direct-ablation/in.slab-direct-yhi.verify,,no
3,slab-isthmus-finite-surface-verification,tests/inputs/slab-isthmus-ablation/in.slab-isthmus-finite-surface.verify,i
4,slab-isthmus-ghost-wall-verification,tests/inputs/slab-isthmus-ablation/in.slab-isthmus-ghost-wall.verify,isthmus,n
5,sphere-isthmus-local-deletion-verification,tests/inputs/sphere-isthmus-ablation/in.sphere-isthmus-local-deletion.ve
6,sphere-isthmus-normal-carryover-verification,tests/inputs/sphere-isthmus-ablation/in.sphere-isthmus-normal-carryove
7,sphere-isthmus-kinetic-theory-verification,tests/inputs/sphere-isthmus-ablation/in.sphere-isthmus-kinetic-theory.ve
8,sphere-isthmus-normal-carryover-convergence,tests/inputs/sphere-isthmus-ablation/in.sphere-isthmus-normal-carryover
```

The runner auto-detects optional features from the CMake build cache. Cases with unmet requirements are skipped when selecting `all`. Cases marked `expected-fail` are allowed to return a verification failure, and their CSV data is still included in the generated report.

30.3 Build The Combined Report

Run all report cases and build one combined PDF:

```
cmake --build build --target test-report
```

The generated files are:

```
build/output/test-report.tex
build/output/test-report.pdf
build/output/test-report-data/<test-name>/report.csv
```

The report includes:

- a table of contents for jumping to individual cases and input listings;
- a global summary table with case name, quantity, error, tolerance, norm, and pass/fail;
- one section per test case, starting on a new page;
- the test input listing for each case;
- any input files included by the test input, also starting on new pages;
- actual vs exact plots for each verified quantity;
- error plots with tolerance lines for each verified quantity;
- convergence order tables for convergence cases;
- convergence plots with each grid resolution shown in the legend.

30.4 Direct Tool Use

The report runner accepts the same selection styles we expect to use as the suite grows:

```
python3 tools/run-test-report.py all
python3 tools/run-test-report.py 1-4
python3 tools/run-test-report.py slab-direct-command-verification
python3 tools/run-test-report.py slab-direct-command-verification,slab-isthmus-ghost-wall-verification
```

Optional features can be overridden explicitly:

```
python3 tools/run-test-report.py all --available isthmus
python3 tools/run-test-report.py all --available ""
```

Names can be shortened if the partial name matches exactly one case:

```
python3 tools/run-test-report.py fix-verification \
  --out build/output/fix-report.tex
```

The lower-level report builder can still aggregate already-written CSV files:

```
python3 tools/build-test-report.py --discover build/output/test-report-data \
  --out build/output/discovered-report.tex --pdf
```

Discovery mode does not know input files unless named cases are provided, so the normal workflow uses `tools/run-test-report.py`.

30.5 Convergence Reports

When a test input uses the `convergence` command, the executable writes extra CSV files next to the normal `report.csv`:

```
convergence.csv
convergence-order.csv
convergence-<quantity>-<case>.csv
```

The report generator uses these files to add a convergence order table and plots. The table records the first and last refinement values, the corresponding errors, the measured order, the allowed order band, whether monotone error reduction was required, and the pass/fail result. The plots show the convergence error points and actual/exact time histories for each resolution with legend entries identifying the varied input values.

31 Documentation

Documentation is part of the development loop. Whenever a command or behavior is added or edited, update the relevant Markdown page in `docs/` in the same change.

31.1 Source Format

The manual source is plain Markdown. Command pages live in:

```
docs/commands/
```

Concept pages live in:

```
docs/concepts/
```

Examples live in:

```
docs/examples/
```

Development notes live in:

```
docs/development/
```

The web documentation layout is described by:

```
mkdocs.yml
```

The intended web-docs stack is MkDocs Material because it is modern, searchable, widely used, and still keeps every page as readable Markdown.

31.2 PDF Manual

The repository also has a local PDF builder:

```
cmake --preset standalone
cmake --build --preset docs
```

It writes:

```
docs/isthmus-ablation-core-manual.pdf
```

The lower-level builder is:

```
python3 tools/build-docs-pdf.py
```

This PDF builder is intentionally lightweight. It is a dependable review artifact for local development, while MkDocs Material remains the richer web documentation path.

31.3 Update Rule

When functionality changes:

- update the command reference for syntax changes;
- update concepts if the architecture or verification model changes;
- update examples when user-facing input files change;
- rebuild the PDF manual before handing the change back.